

FI360 TYRE MODULE

DEVELOPER BACKLOG, TEST PLAN & TEST CASES

Fleet Intelligence 360 (FI360) — Version 1.0
Fleet Associates Africa Ltd (FAA)
10 August 2026

Document Control

Field	Value
Document ID	FI360-DEV-TYR-008
Version	1.0
Status	Development Baseline
Parent Documents	FI360 Tyre SRS; Database/Data Dictionary; API & Integration Contract; UI/UX Specification
Module	FI360 Tyre Management
Purpose	Convert approved requirements into executable development and testing work

1. Purpose

This document translates the approved FI360 Tyre requirements into a development backlog and verification framework. It is intended for the Product Owner, Solution Architect, UI/UX designer, frontend team, backend/API team, database engineer, QA team, DevOps and UAT stakeholders.

The module is developed as an independently usable product capability but must remain integration-ready. The FI360 development standard requires modular independent operation, API-first integration, event-driven readiness, shared security/identity and separate module boundaries. [filecite](#)[turn1file3](#)[L312-L330](#)

2. Development Principles

- Build against the approved SRS, data dictionary and API contract.
- Do not introduce UI fields or business rules without traceability to an approved requirement or change request.
- Keep Tyre operational when optional modules are absent.
- Use canonical FI360 identifiers and shared identity/tenant controls.
- Enforce authorization at the API/backend, not only in the UI.
- Record auditable material transactions.
- Use versioned APIs and stable event contracts.
- Design for future Workshop/Fleet/Telematics/Analytics integration without creating hard dependencies.

3. Technology Alignment

Layer	Recommended Technology Direction
Frontend	TypeScript + React/Next.js, using the FI360 Core design system
Backend	TypeScript with NestJS or an equivalent enterprise service framework

Database	PostgreSQL
ORM/Data Access	Prisma or approved equivalent
API	REST/JSON, versioned
Async Integration	Event bus/message broker using FI360 Core standards
Authentication	OIDC/OAuth2-compatible identity service
Deployment	Containerized services with CI/CD
Observability	Centralized logs, metrics and tracing

These are implementation recommendations for this execution document. The final technology baseline must remain consistent with the approved FI360 architecture and enterprise development standards.

4. Definition of MVP

The Tyre MVP should establish the complete operational lifecycle rather than attempting every advanced analytics capability.

MVP Area	Included
Master Data	Tyre registration, specifications, identifiers
Stock	Receiving, location and movement
Fitment	Fitment and position management
Inspection	Tread, pressure and condition
Operations	Removal and rotation
Lifecycle	Repair, retread and disposition
History	Complete tyre timeline
Alerts	Rule-based exceptions
Security	Roles and permissions
Reporting	Core operational reports
Integration	API-ready interfaces and event readiness

5. Epics

Epic ID	Epic	Outcome
TYR-E01	Tyre Master & Registration	Every physical tyre can be uniquely identified and managed
TYR-E02	Stock & Location	Tyres can be received, stored, moved and issued
TYR-E03	Fitment & Position	Tyres can be safely fitted and associated with asset positions
TYR-E04	Inspection	Tyre condition can be measured and evaluated
TYR-E05	Removal & Rotation	Operational tyre changes are traceable
TYR-E06	Repair & Retread	Lifecycle interventions are recorded
TYR-E07	Lifecycle & Disposition	End-of-life decisions are controlled
TYR-E08	Alerts & Exceptions	Operational risks are surfaced
TYR-E09	Dashboard & Reporting	Users can understand performance and cost
TYR-E10	Security & Audit	Access and material actions are controlled
TYR-E11	API & Integration	Tyre is independently usable and integration-ready
TYR-E12	Quality & Deployment	Module is tested, observable and deployable

6. Feature Backlog

ID	Feature	Priority
TYR-F01	Tyre registration	P0
TYR-F02	Tyre search/register	P0
TYR-F03	Tyre details & timeline	P0
TYR-F04	Receiving & stock	P0
TYR-F05	Location movement	P0
TYR-F06	Fitment	P0
TYR-F07	Removal	P0
TYR-F08	Inspection	P0
TYR-F09	Rotation	P1
TYR-F10	Repair	P1
TYR-F11	Retread	P1
TYR-F12	Retirement/scrap	P1
TYR-F13	Alerts	P1
TYR-F14	Dashboard	P1
TYR-F15	Reports	P1
TYR-F16	Permissions	P0
TYR-F17	Audit trail	P0
TYR-F18	API/event integration	P0
TYR-F19	Observability	P0
TYR-F20	UAT/admin controls	P0

7. User Stories — Master Data

Story	User Story	Acceptance Summary
TYR-US-001	As a tyre officer, I want to register a tyre so that it has a unique FI360 identity.	Unique identifier; required data validated; audit recorded.
TYR-US-002	As a tyre officer, I want to scan a tyre so that I reduce manual entry.	Barcode/QR resolves correct tyre or provides registration path.
TYR-US-003	As a manager, I want to search tyres so that I can locate records quickly.	Search by approved identifiers and filters.
TYR-US-004	As an authorized user, I want to view tyre history so that I can trace lifecycle events.	Timeline is chronological and permission controlled.

8. User Stories — Operations

Story	User Story	Acceptance Summary
TYR-US-005	As a stores officer, I want to receive tyres into stock.	Receipt and location recorded.
TYR-US-006	As a stores officer, I want to move a tyre between locations.	Movement is validated and audited.
TYR-US-007	As a technician, I want to fit a tyre to an asset position.	Tyre, asset and position are validated before posting.
TYR-US-008	As a technician, I want to remove a tyre with a reason.	Reason and destination are mandatory.
TYR-US-009	As a technician, I want to rotate tyres.	Before/after positions are recorded.
TYR-US-010	As a technician, I want to inspect a tyre.	Measurements, condition and recommendation are recorded.

9. User Stories — Lifecycle

Story	User Story	Acceptance Summary
TYR-US-011	As a tyre supervisor, I want to record repairs.	Repair type, result and relevant cost/history captured.
TYR-US-012	As a tyre manager, I want to record retreading.	Casing/cycle/supplier/result are traceable.
TYR-US-013	As an authorized user, I want to retire or scrap a tyre.	Controlled reason, approval and disposition are recorded.
TYR-US-014	As a manager, I want lifecycle cost visibility.	Configured cost components aggregate correctly.

10. User Stories — Intelligence & Reporting

Story	User Story	Acceptance Summary
TYR-US-015	As a manager, I want tyre exceptions visible on the dashboard.	Critical/warning exceptions are prioritized.
TYR-US-016	As a manager, I want operational reports.	Reports support approved filters and export permissions.
TYR-US-017	As a manager, I want tyre performance information.	Approved KPIs calculate from controlled data.
TYR-US-018	As an auditor, I want an audit trail.	Material changes identify actor, time and event/reference.

11. User Stories — Integration

Story	User Story	Acceptance Summary
TYR-US-019	As a Tyre-only customer, I want the module to operate independently.	No mandatory dependency on Workshop/Fleet/etc.
TYR-US-020	As a customer adding Workshop later, I want Tyre events to connect to Workshop.	Contextual links/actions appear when licensed and authorized.
TYR-US-021	As an integrator, I want stable APIs.	Versioned contracts and validation are enforced.
TYR-US-022	As an integration engineer, I want events.	Material events can be published according to FI360 standards.

12. Technical Work Breakdown

Workstream	Tasks
Architecture	Confirm module boundary; service responsibilities; integration points
Database	Tables, keys, indexes, constraints, migrations, seed/reference data
Backend	Domain services, validation, transactions, authorization, audit
API	Endpoints, DTOs, error model, pagination, filtering, versioning
Frontend	Routes, screens, forms, tables, dashboard, responsive states
Integration	Events, external references, idempotency, retries
Security	RBAC, tenant isolation, authorization tests, audit
QA	Unit, integration, system, regression, performance, security, UAT

DevOps	CI/CD, environment configuration, migrations, monitoring, rollback
--------	--

13. Database Development Tasks

- Implement Tyre master record and controlled reference data.
- Implement lifecycle event/history model.
- Implement fitment and position relationship.
- Implement inspection records and measurement structures.
- Implement stock/location relationships.
- Implement repair/retread/disposition records.
- Implement audit/event metadata.
- Create unique and foreign-key constraints.
- Create indexes for identifiers and high-volume operational queries.
- Create migration and rollback strategy.
- Prepare test fixtures and controlled seed data.

14. Backend/API Tasks

- Implement domain validation before persistence.
- Implement transactional operations for fitment/removal/rotation.
- Implement authorization checks.
- Implement idempotency for retry-sensitive transactions.
- Implement pagination/filtering/sorting.
- Implement standardized error responses.
- Implement API versioning.
- Implement audit recording.
- Implement event publication after committed transactions.
- Implement integration status handling without falsely reporting external success.

15. Frontend Tasks

- Build FI360 Core application shell integration.
- Implement Tyre navigation.
- Implement dashboard.
- Implement register/search/details screens.
- Implement fitment/removal/inspection workflows.
- Implement stock and location screens.
- Implement lifecycle screens.
- Implement alerts and reports.
- Implement responsive/mobile layouts.
- Implement loading/empty/error/offline states.
- Map all UI fields to approved API contracts.

16. API Test Strategy

Test Area	Checks
Contract	Request/response schema, required fields, types
Authentication	Valid/invalid identity

Authorization	Role/permission enforcement
Validation	Invalid business/technical values rejected
CRUD	Create/read/update behaviour
Transactions	Atomicity and consistency
Concurrency	Conflicting updates handled safely
Idempotency	Retries do not duplicate transactions
Pagination	Stable and correct result sets
Errors	Standardized error codes/messages
Integration	Events and downstream status

17. Unit Test Plan

- Domain validation rules.
- Lifecycle state transitions.
- Fitment eligibility.
- Position validation.
- Inspection measurement validation.
- Cost calculations.
- Alert rule evaluation.
- Permission decisions.
- API DTO validation.
- Utility/helper functions.

Target coverage should be agreed by the engineering team; critical domain and security logic should receive the highest coverage rather than pursuing a single blanket percentage.

18. Integration Test Plan

- Frontend → API authentication.
- Frontend → API authorization.
- API → database transaction integrity.
- Tyre → event publication.
- Tyre → optional Workshop integration.
- Tyre → optional Fleet/Asset integration.
- Tyre → notification/alert service.
- Report queries against realistic volumes.
- Failure/retry scenarios.

19. System Test Plan

Scenario	Expected Result
Register tyre	Tyre exists with valid identity and audit
Receive tyre	Stock and location are correct
Fit tyre	Asset/position relationship is correct
Inspect tyre	Measurements and recommendation persist
Remove tyre	Fitment ends and destination is recorded
Rotate tyre	Old/new positions are correct
Repair tyre	Repair history and status are correct
Retread tyre	Cycle and result are correct
Retire tyre	Lifecycle ends according to policy
Search	Correct records returned
Dashboard	KPIs match source data

Report	Filters and totals are correct
--------	--------------------------------

20. Security Test Plan

- Tenant isolation: one customer cannot access another customer's records.
- Role isolation: unauthorized actions are rejected server-side.
- Object-level authorization: IDs cannot be used to bypass permissions.
- Audit integrity: material transactions leave traceable records.
- Authentication failure handling.
- Session/token expiry handling.
- API input validation and injection resistance.
- Sensitive data exposure review.
- Export authorization.
- Administrative action controls.

21. Performance Test Plan

Test	Target Principle
List/search	Responsive at expected production data volume
Tyre details	Acceptable response with long lifecycle history
Fitment transaction	Consistent under concurrent users
Inspection submission	Stable under operational load
Dashboard	Efficient aggregation/caching strategy
Reports	Long-running reports must not block normal operations
API throughput	Measure against agreed service-level targets
Database	Indexes and query plans validated

22. UAT Test Cases

UAT ID	Scenario	Expected Result
UAT-01	Register a new tyre	Tyre successfully registered
UAT-02	Search by serial	Correct tyre displayed
UAT-03	Receive into stock	Correct location/status
UAT-04	Fit to vehicle position	Fitment recorded
UAT-05	Inspect tyre	Condition/measurements recorded
UAT-06	Remove tyre	Reason/destination recorded
UAT-07	Rotate tyres	Positions updated correctly
UAT-08	Record repair	Repair history available
UAT-09	Record retread	Retread history available
UAT-10	View lifecycle	Complete timeline visible
UAT-11	Review alert	Exception and action visible
UAT-12	Run report	Correct filtered output
UAT-13	Unauthorized action	Action rejected
UAT-14	Tyre-only operation	Module remains fully usable
UAT-15	Workshop added later	Relevant Tyre/Workshop links work without breaking Tyre

23. Regression Test Suite

- All P0 workflows after every release candidate.
- Permission regression after security changes.
- API contract regression after endpoint changes.

- Database migration regression.
- Search/filter regression.
- Mobile/responsive regression.
- Report calculation regression.
- Event/integration regression.
- Audit regression.

24. Defect Classification

Severity	Meaning
S1 Critical	Security breach, data corruption, unusable core workflow or release blocker
S2 High	Major business workflow failure with no acceptable workaround
S3 Medium	Important defect with workaround
S4 Low	Minor UI, wording or non-critical behaviour issue

25. Definition of Ready

- User story has clear business value.
- Acceptance criteria are testable.
- UI/UX reference exists where applicable.
- API/data dependencies identified.
- Security implications identified.
- Required reference data identified.
- No unresolved ambiguity that blocks development.

26. Definition of Done

- Code reviewed.
- Automated tests pass.
- Relevant integration/system tests pass.
- Security checks pass.
- Database migration tested.
- API contract validated.
- UI states completed.
- Audit behaviour verified.
- Observability added.
- Documentation updated.
- UAT acceptance obtained for release scope.
- Deployment and rollback path verified.

27. Sprint Structure

Sprint	Focus
Sprint 0	Architecture, environments, repositories, CI/CD, Core integration
Sprint 1	Tyre master, registration, search, details
Sprint 2	Stock, receiving, location movement
Sprint 3	Fitment, position, removal
Sprint 4	Inspection and alerts

Sprint 5	Rotation, repair, retread
Sprint 6	Lifecycle/disposition, dashboard, reports
Sprint 7	Integration, security hardening, performance
Sprint 8	UAT, defect closure, release readiness

Sprint duration should be agreed by the delivery team; the sequence is a dependency-oriented baseline rather than a fixed calendar commitment.

28. Release Gates

Gate	Required Evidence
Gate 1 — Architecture	Approved architecture, repository and CI/CD
Gate 2 — MVP Feature Complete	P0 features implemented
Gate 3 — QA Ready	Automated/unit/integration tests passing
Gate 4 — Security Ready	Security test results acceptable
Gate 5 — UAT Ready	System test pass and UAT environment stable
Gate 6 — Production Ready	UAT acceptance, monitoring, rollback and deployment plan

29. Module Integration Readiness

The Tyre module should publish integration-ready business events and expose stable API contracts without requiring other modules for its own operation. When another FI360 module is later purchased, integration should be activated through configuration/entitlements rather than rewriting the Tyre core.

Future Module	Tyre Connection
Workshop	Work orders, repairs, maintenance actions
Fleet	Canonical Asset and vehicle context
Fuel	Potential cost/usage correlation
Telematics	Mileage/hours and operational context
Analytics	Cross-module KPIs and benchmarking
Finance	Cost/accounting context
Parts	Repair/maintenance parts relationship

30. Traceability Matrix

Requirement Area	Development Evidence	Test Evidence
Tyre identity	Master data + API + UI	Registration/search tests
Fitment	Fitment service/API/UI	Fitment/authorization tests
Inspection	Inspection service/UI	Validation/system tests
Lifecycle	Lifecycle services/UI	Transition tests
Audit	Audit service/database	Audit integrity tests
Permissions	Identity/RBAC/API	Security tests
Integration	API/events	Contract/integration tests
Reporting	Reporting layer/UI	Report/UAT tests

31. Risks & Controls

Risk	Control
Tyre becomes dependent on future modules	Strict module boundary and optional integration
Duplicate tyre identity	Unique constraints and validation
Incorrect fitment	Position/specification validation + confirmation
Data corruption during lifecycle actions	Transactions + audit
Unauthorized changes	Server-side authorization
Poor field usability	Mobile-first task prototypes and UAT

Slow reports	Asynchronous/reporting strategy and indexed queries
Integration failures	Retries, idempotency and integration status
Untraceable changes	Audit events for material transactions

32. Delivery Team Responsibilities

Role	Primary Responsibility
Product Owner	Priorities, scope, acceptance
Solution Architect	Architecture and boundaries
Backend Engineer	Domain/API implementation
Frontend Engineer	UI implementation
Database Engineer	Schema/performance/migrations
QA Engineer	Test automation/system testing
Security Engineer	Security verification
DevOps Engineer	CI/CD, deployment, observability
UX Designer	Prototype, usability and design system
UAT Users	Operational acceptance

33. Immediate Development Sequence

1. Freeze approved SRS/data/API/UI baselines.
2. Create repository/module structure.
3. Create database migrations and seed data.
4. Implement authentication/authorization integration.
5. Implement Tyre master and registration.
6. Implement search/details/timeline.
7. Implement stock/location.
8. Implement fitment/removal.
9. Implement inspection/alerts.
10. Implement lifecycle features.
11. Implement dashboard/reports.
12. Implement integrations and events.
13. Execute QA/security/performance.
14. Execute UAT and release.

34. Next Document

DOCUMENT 09 should be the FI360 TYRE MODULE IMPLEMENTATION & DEPLOYMENT BLUEPRINT. It should convert this backlog into the actual technical build environment: repository structure, services, database migration sequence, API folder structure, frontend folder structure, environments, CI/CD, Docker/container strategy, configuration, observability, deployment, backup/restore, rollback and production support.

This creates the bridge from documentation into the engineering environment and prevents the project from remaining only a collection of specifications.

FI360 development principle: build each module independently, but never build it in isolation.